Resolving Project Intent into a Kubernetes Deployment Model

Project Task 2: Model Transformations
Project Type: Code Generation
October 16, 2026

Joris Jonkers (s1829157)

Model Driven Engineering
University of Twente

Contents

1 Purpose of the Project **1**

1.1 Goals . . . 1
1.2 Role of the Transformations . . . 1
1.3 Refinement of Tasks 0 and 1 . . . 2
1.4 Assumptions and Scope . . . 2

2 Alternatives and Justification **3**

2.1 Existing Kubernetes Modelling Tools . . . 3
2.2 The Transformation Language . . . 4
2.3 The Structure of the Transformation . . . 4

3 Solution Design **5**

3.1 The Flattening . . . 6
3.2 Interface, Mappings and Queries . . . 6
3.3 Worked Example . . . 7
3.4 The Derivation Rules . . . 8
3.5 The Trace and End Sections . . . 11
3.6 The Black-Box Library . . . 12
3.7 The Final Pass . . . 12
3.8 Aborts and Running the Transformations . . . 12

4 Tests and Evaluation **13**

4.1 Results . . . 13
4.2 Limitations . . . 14
4.3 Reflection . . . 15

A Changes to the Task 1 Metamodels **16**

A.1 The Pinned Inputs Metamodel . . . 16
A.2 Changes to the Target Metamodel . . . 16
A.3 Changes to the Source Metamodel . . . 18
A.4 Grammars for the Pinned Inputs . . . 19

1 Purpose of the Project

1.1 Goals

The project turns short deployment documents into Kubernetes files for an existing small cluster. Each project repository holds one `Project Intent` document describing its `Application`s: their `Process`es, memory and CPU limits, ports, dependencies and external routes. One `Platform Intent` document defines shared policies and resources, including cluster entry points, health-check timings and backup policies. Task 1 defined two Ecore metamodels: a source metamodel for the authored documents and a target metamodel for the `Resolved Deployment`. The target model records the values derived for each `Application` and the paths of the files to be generated.

Task 2 implements the transformation to `Resolved Deployment`. It combines all authored models with recorded cluster facts, image digests and other pinned inputs, producing one resolved model per `Project`. For example, an image name becomes a repository address pinned to a digest, and a dependency declaration becomes a host and port. Task 3 then generates files from these resolved values.

Term	Meaning
`Process`	One program of an `Application`, running in its own pods.
`Dependency edge`	A declaration by one `Process` that it calls a named port of another `Application`.
`Pinned input set`	The read-only inputs besides authored documents: node contract (machines and capacity), images lock (image digests), `ClusterState` snapshot, migration proofs and configuration-file contents.
`Asset`	A configuration file mounted into a `Process`.
`Exposure`	An external address and routes to an `Application`'s ports.
`Switchover`	How a new version replaces the old: blue/green (both run; traffic switches once the new is healthy), rolling, or stop-start.
`ReleaseGate`	The resolved configuration for the service that blocks a blue/green switch until the `Application`'s `Process`es are healthy.
`ReconcileUnit`	A group of resources that Flux reconciles together, with dependencies on other such groups.
`Entry point`	A platform listener that admits specified audiences and forwards routes through a proxy; class `Tier` in the source metamodel.
`Revision`	A release identifier computed by hashing an `Application`'s resolved configuration.
`Estate`	All `Project`s, the `Platform Intent` and the `Pinned input set` read by one run.
`Flattened model`	`Project Intent` with every shared declaration copied down onto the `Process`es that hold it (class `EffectiveProject`).

Table 1: Project Terms Used in This Report

Constraints. Task 2 requires ATL or QVT-Operational. The transformation must give the same result for the same pinned inputs, without reading live cluster state. It must reproduce the oracle files supplied with the specification's examples: expected resolved values, dependency edges and generated files, including byte-exact JSON and release digests. If a value cannot be derived, the run must abort with an error message. A missing or fabricated value could otherwise produce a valid-looking file that deploys the wrong configuration.

The resolved model must also contain every value needed by the Task 3 templates, including paths, ports and names. Otherwise, tests of that model could pass while a template made an unchecked deployment decision.

1.2 Role of the Transformations

Figure 1 shows the pipeline. Xtext converts each document to a source model, and the Task 1 OCL constraints reject invalid ones. Three QVT-Operational transformations then run in sequence.

The first transformation, the flattening, copies declarations made once for an entire `Project` or `Application` down onto each `Process`. Its output is the flattened model, which still conforms to the source metamodel. Although it was implemented in Task 1, it was not reported there, so Section 3.1 covers it here. The second

transformation, the resolution, reads the flattened models and the `Pinned input set`, the additional read-only inputs, and derives one complete `Resolved Deployment` per `Project`. It is the main transformation and the subject of this report. The third, the final pass, collects Vault policies from all resolved models to plan one job that installs them in Vault, the platform's secret management service. This requires every project's output, so it runs after the individual resolutions (Section 3.7).

A small Java exporter writes the resolved dependency edges as JSON for comparison against the oracle. The Acceleo templates of Task 3 then read the `Resolved Deployments` and generate the Kubernetes files.

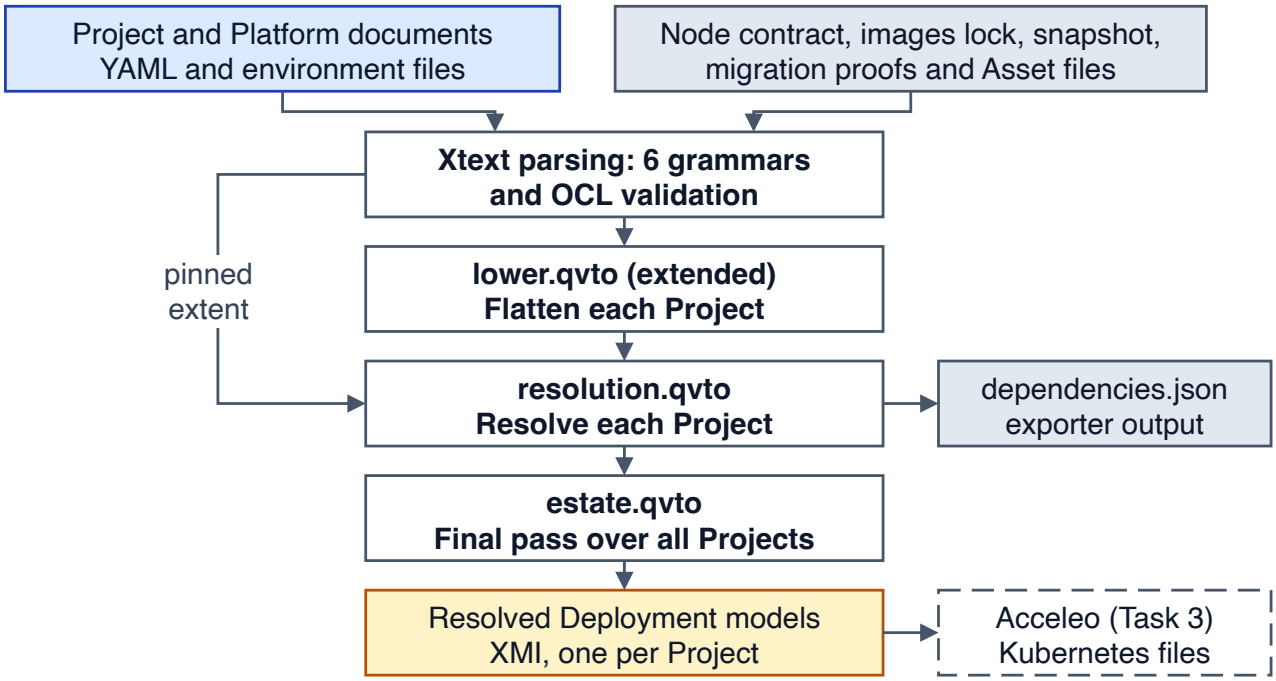


Figure 1: The Pipeline, Including the Flattening Extended and the Pinned-Input Parsers Added in Task 2

1.3 Refinement of Tasks 0 and 1

Task 0 treated the node contract (the list of machines and their capacity), the images lock (each image pinned to a digest) and the `ClusterState` snapshot as plain inputs, while Task 1 only modelled the two authored documents. The resolution needs these as models, so a third metamodel now describes the `Pinned input set`, with its own Xtext grammar for each document (Appendix A.1). A separate package keeps these supporting inputs distinct from the deployment language used by `Project` authors.

Task 1 described three stages: authored, resolved and generated text. The flattening now resolves shared declarations before resolution: each `Process` receives the declarations that apply to it from its `Project` and `Application`. Resolution rules therefore need not repeat that merge logic (Section 2.3.1).

Task 1 planned one resolved model for the entire cluster, because resolving dependency edges, routes and network access requires a cluster-wide view. Every run still reads every `Project`. The output is now one `Resolved Deployment` per `Project`, because each model then stays small and has one oracle to compare with (Section 2.3.2). Each `Application` still has its own `ResolvedApplication` inside that model. The final pass adds the one job that no single `Project`'s resolution can compute (Section 3.7).

Task 1 demonstrated the target metamodel with only `minimal`. The other examples needed additional concepts: migrations, backup service accounts, API access, sidecar containers and twelve more file types. Appendix A lists every change.

1.4 Assumptions and Scope

The transformation assumes four conditions.

- **Checked input.** The OCL constraints check the documents before transformation. Aborts reject invalid inputs that these constraints do not yet catch (Section 3.8).

- **One cluster per run.** Each execution of the pipeline reads all documents for one cluster, including one `Platform Intent` and one node contract that name the same cluster. All concepts, examples and oracles derive from that single existing cluster.
- **One reading of the specification.** If several providers satisfy a dependency, the transformation uses the deterministic name order recorded in the oracles. The comparisons check this interpretation; they do not establish that it is the only valid one.
- **No live cluster access.** The transformation only reads the documents provided to it. Any fact known only to a running cluster, such as which node holds a particular volume, must arrive via the `ClusterState` snapshot captured by another process.

More Than One Cluster. A second cluster would need a separate run with its own `Platform Intent`, node contract and `ClusterState` snapshot. Environment overlays are already selected by cluster name (Section 3.4.3). The model would also need to declare which clusters each `Project` targets and to represent cross-cluster network access with address ranges. Such dependencies could use the external-provider mechanism already used by `auth` for its mail server (Section 3.4.2); this extension has not been implemented.

Table 2 sets the scope of Task 2.

In scope	Out of scope
The resolution of eight examples: `minimal`, `auth`, `data`, `delivery`, `edge`, `knowledge-platform`, `observability` and `secrets`.	The `knowledge` example, which is being retired with its project and is excluded from this test set (Section 4.2).
Every derivation these examples use, from dependency edges and placement to backups, migrations, network peers and the path of every generated file.	The text of the generated files, which is Task 3.
The flattening's extensions, the final pass, and the `Pinned input set` metamodel and grammars.	Applying files to a running cluster or targeting multiple clusters.

Table 2: Scope of Task 2

2 Alternatives and Justification

2.1 Existing Kubernetes Modelling Tools

Three open-source tools were reviewed to see whether they could produce the `Resolved Deployment` from `Project Intent` and `Platform Intent`, or at least take part in the pipeline. Table 3 compares them.

Tool	What it does	Why it cannot replace the resolution	Where it could help
KubeDiagrams [1]	Draws diagrams from Kubernetes files or a live cluster, mapping kinds to icons.	It needs Kubernetes resources as input, so it cannot derive them from deployment intent.	It could visualize the generated files to show links produced by a `Project` document.
KubeOrch [2]	A drag-and-drop editor whose back end wires ports, fills templates, writes files and deploys.	It has no metamodel and no constraints; its rules are Go code, and it deploys directly, not via Flux.	Its graph notation could inform a `Project` document editor.
kubert [3]	A command-line tool to switch cluster contexts and run commands on multiple clusters.	It is an operations tool and does not model or generate anything.	It could apply files to several clusters, which is out of scope.

Table 3: Existing Kubernetes Modelling Tools

KubeOrch comes closest: it turns service links into addresses and ports. However, it has no shared platform policy or pinned-input model from which to derive placement, locked images or backups, and no resolved

intermediate model to test. KubeDiagrams could inspect the Task 3 output, while a KubeOrch-style editor could author `Project Intent` using the same Ecore metamodel as the transformation.

2.2 The Transformation Language

Task 2 permits ATL or QVT-Operational (QVTo). Both can combine several input models and maintain source-to-target traces; neither capability alone distinguishes them. QVT-Relations and Java with a rule library such as SiTra were also considered as alternative approaches.

QVTo was selected for the dependencies between construction steps. A `Process`'s placement combines its declarations with the node contract; a route refers to an already resolved `Process`; and an `Application`'s `Revision` hashes its complete resolved representation. Ordered mapping calls construct the `Process`es before their routes, trace lookup reuses those target objects, and an `end` section reads the assembled model to compute the `Revision`. Typed input extents define the documents searched by OCL queries. This keeps construction, cross-reference lookup and final derivation in one resolution.

ATL remains a viable alternative. Its matched rules can match several source elements and resolve references through automatic traces, while lazy, unique lazy and called rules support explicit invocation. The distinction is therefore how the final derivation is expressed. In ATL's usual execution model, target models are write-only: hashing a completed `Application` would require deriving its canonical representation from source helpers or adding a pass that reads the generated model. QVTo allows the completed target objects to be inspected within the same transformation. The cost is that mapping order becomes an explicit responsibility, rather than being delegated to ATL's matched-rule engine.

QVT-Relations can also express a one-way transformation. Its consistency checking and multidirectional execution are unnecessary here, since the resolved model is regenerated from intent. QVTo was preferred because the construction order and the point at which hashing becomes valid can be stated directly. Java offers the same control and standard debugging tools. Plain Java would need a trace data structure, whereas SiTra supplies one; neither lacks the required expressive power. QVTo retains OCL navigation over the existing metamodels for both constraints and derivations, avoiding a separate Java rule implementation for this proof of concept. SHA-256 is delegated to a Java black box (Section 3.6).

Speed was not a selection criterion. The compiler, which derives a resolved model and subsequently generates deployment files, is a proof of concept for checking the design against the specification's examples. Its design will later be ported to a general-purpose language such as TypeScript or Rust.

2.3 The Structure of the Transformation

Every derived value must be recorded in a model before code generation. This allows tests to compare the values with the oracles independently of the templates that write them to files.

2.3.1 Flattening Before the Resolution

A single transformation from authored model to `Resolved Deployment` would work, but a flattening precedes the resolution. The flattening copies shared declarations down to each `Process`. For example, a `Project` startup budget of 30s reaches every `Process` except one that declares a more specific value, such as 20s. Resolution can then read the selected budget directly from each `Process`, without searching its ancestors. QVT specification Section 8.1.13 likewise composes a preparation transformation before the main one [4], but the reason for this split is to isolate declaration precedence from deployment derivation. Its cost is an additional model and the `EffectiveProject` and `EffectiveApplication` classes, whose copying and override behaviour need their own tests.

2.3.2 Resolving One Project with All Projects as Input

One resolution over all `Project`s at once could produce one `Resolved Deployment` for the whole estate. Instead, each invocation resolves one `Project` into its own model while reading every `Project`'s documents from the same input extent. This preserves a separate output and oracle comparison for each `Project`, while the shared input supplies the cluster-wide view needed for dependencies, routes and network access. It does not make a `Project` independently resolvable: a change elsewhere can affect its output. Values that depend on all resolved outputs, including the digest over the Vault documents, require the final pass over those models (Section 3.7).

A single estate-wide output could compute these values in the same invocation but would combine the output and comparison units.

2.3.3 Typed Extents

A rule also has to find the run's other documents. An unqualified `allInstances()` lookup does not identify which input model is intended, particularly when authored and flattened `Processes` are both loaded. Instead, the transformation selects roots by type from the named `intent` and `pinned` extents and stores them in transformation properties such as `projects` and `lock`. Rules navigate from these properties, making their input scope explicit (Section 3.2).

2.3.4 Rejecting Unresolvable Inputs

A default for a value the resolution cannot derive would give a file that looks valid and deploys the wrong configuration, so the run aborts instead, through a fatal assertion in an abort query (Section 3.8). The caller then has to handle the failure.

3 Solution Design

Task 2 uses three QVT-Operational transformations. The flattening (`lower.qvto`) is about 340 lines with 37 mapping operations and 8 queries. The resolution (`resolution.qvto`), the main transformation, is about 1700 lines with 8 mapping operations and 140 queries. The final pass (`estate.qvto`) is about 130 lines with no mappings but one helper.

Figure 2 shows the metamodels each transformation reads and writes. The three differ in kind, and all are one-way.

- The flattening is endogenous and horizontal: it reads and writes the source metamodel at the same abstraction level, producing a new model. One `Project` goes in, one comes out.
- The resolution is exogenous and vertical, a refinement: it reads source and pinned-input metamodels and writes the target metamodel, taking several input models and producing one output.
- The final pass is endogenous and in-place: it updates the resolved models of a run through an `inout` extent.

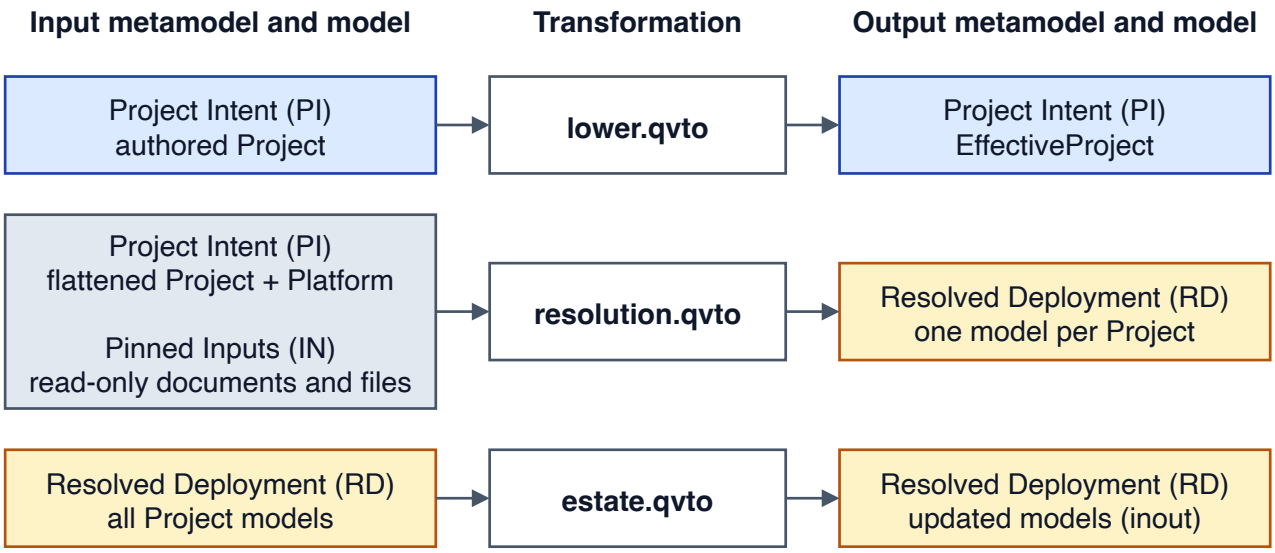


Figure 2: The Three Transformations and Their Metamodels

3.1 The Flattening

The flattening, `lower.qvto`, turns each authored `Project` into an `EffectiveProject`. Every `Process` carries all shared declarations that apply to it, while the `Project` and `Application` carry none. Task 2 extended the Task 1 implementation with API access and engine `Grants` (declared access to a secret of a database or transit engine).

Its 37 mappings are nearly all simple copies; its queries do the merging. A list such as `secrets`, dependency edges, configuration files or environment entries becomes the union of declarations from `Project`, `Application` and `Process` levels. If several levels declare the same item, the most specific declaration wins. A single value, such as the startup budget or cutover (whether the `Application` keeps serving during an update), also comes from the most specific level that declares it. Memory and CPU remain only on the `Process`.

A shared declaration needs a separate copy in each receiving `Process`. Reusing a cached mapping result would move that object to the last `Process`, because an EMF object can have only one container. Inside `Process::lower`, `self` is the receiving source `Process`; passing it to each declaration's `copyFor(self)` makes the cache distinguish the required copies. For migrations, one disjuncts mapping dispatches to the copy operation for the source subtype: managed, self-managed or no migration. The caller can copy any migration without repeating that subtype selection.

The flattening has no aborts of its own: it only copies and merges declarations after input checks. Missing values needed for deployment, such as an unlocked image, are rejected later by the resolution.

3.2 Interface, Mappings and Queries

Example 1 shows how the inputs are grouped: `intent` contains the deployment declarations, `pinned` contains their supporting inputs, and `resolved` receives the target model.

```
transformation resolution(in intent : PI, in pinned : IN, out resolved : RD)
        access library Hashing;
configuration property project : String;     -- the project this run resolves
property projects : Sequence(EffectiveProject) =
        intent.rootObjects()[EffectiveProject]->sortedBy(p | p.project)->asSequence();
property lock : ImagesLock = pinned.rootObjects()[ImagesLock]->any(true);
-- and likewise contract, snapshot, proofs and assetFiles
main() {
        projects->select(p | p.project = project)->any(true).map deployment();
}
```

Example 1: The Signature and Inputs of the Resolution, Abridged

The intent extent holds, for every `Project` in the run, its authored model and its `EffectiveProject`, plus the `Platform Intent`. Authored models remain only because `Platform Intent` references them via Ecore cross-references. Every rule reads the `EffectiveProject` models.

The pinned extent holds the node contract, images lock, cluster-state snapshot, each `Project`'s migration proof (the schema version its migration was tested against), and one `AssetFile` per configuration file. Two configuration properties supply the `Project` name to resolve and the schema package digest to include in the provenance record.

Properties such as `projects` and `lock` select these inputs once; another selects the `Platform Intent`. A dependency lookup navigates through `projects.applications.processes`, so it searches only the flattened models supplied to this run. Sorting `Projects` by name makes the result independent of input file loading order.

Mappings are used for objects that other rules must reference. For example, the trace connects a source `Process` to its `ResolvedProcess`, allowing a route to reuse that result through `resolveone`. A probe belongs only to its parent and needs no such lookup, so a query creates it with an object expression instead. Each mapped target type has one mapping, making the source and result type sufficient to identify the trace entry. Callers filter elements with `->select` before invoking a mapping. No mapping uses a `when` guard or inherits from another.

Table 4 lists the eight mappings and their purpose.

Queries calculate values without modifying the input models. They return scalars such as deadlines, new contained objects such as backup plans, or the target spelling of an enumeration value, for example `businessHours`

Mapping	Creates, and why it is a mapping
`EffectiveProject::deployment`	The `ResolvedDeployment`. Its end section sets values that need the complete model.
`EffectiveProject::unit`	The `Project`'s `ReconcileUnit`, the group Flux applies together. Each `Application` refers to it.
`EffectiveProject::provenance`	The digest of every input and the hash over them. Its end section computes the hash.
`EffectiveApplication::application`	A `ResolvedApplication`. Generated files and the `Revision` refer to it.
`EffectiveApplication::gate`	The `ReleaseGate`'s inputs. The `Application` and its configuration refer to them.
`EffectiveApplication::migrationOf`	The database migration. Its network policy refers to it.
`Process::process`	A `ResolvedProcess`. Gate members, routes, scrapes, policies and generated files refer to it.
`Exposure::exposure`	A `ResolvedExposure`. The generated route file refers to it.

Table 4: Mapping Operations of the Resolution

as `business-hours`. The calling mapping assigns the result. Abort queries terminate the run when a value cannot be derived (Section 3.8). The final pass uses a helper instead because it adds files to an existing output model.

Mappings run in a fixed order. The deployment mapping creates the `Project`'s `ReconcileUnit` and local references to prerequisite units first, then `Application`s, provenance and the path plan, the step that gives every generated file its path. Each `Application` maps its `Process`es before its `Exposure`s, `ReleaseGate` and migration, because those refer to `Process`es through the trace.

3.3 Worked Example

The `minimal` example declares one `Application`, `notes`, with one `Process`, `notes-api`, in about 35 lines of YAML. Table 5 follows declarations through to derived values.

Declared	Rule	Derived in the resolved model
`image: notes-api`	`locked`	`ghcr.io/jorisjonkers-dev/notes/notes-api@sha256:9c1e...`, user/group 1000, from images lock
`startupBudget: 20s`	`deadlineOf`, startup probe	deadline 60s (three times the budget); the startup probe polls the liveness target every 5s and fails after 4 attempts
`cutover: continuous`	`switchoverOf`, `gate`	blue-green; `ReleaseGate` with 1 member, deadline 60s, cadence from the `Platform Intent`, error/latency checks
`memory: 256Mi, cpu: 50m`	`eligible`	all 7 machines in the node contract, each tested for twice the request size for blue/green deployment
`audience: anonymous`	`tierFor`, `exposure`	entry point public-frankfurt with TLS certificates issued through ACME, proxy traefik-public in the `edge-system` namespace
route /, prefix	`precedenceOf`, `middlewareOf`	precedence 1; security headers with strict content policy
`runtime: node`	`profileOf`	environment variables `OTEL_SERVICE_NAME`, `PORT=8080`, `DEPLOYMENT_ENVIRONMENT`, collector address from observability
scrape on http	`ingressOf`, `monitorOf`	ingress from traefik-public and prometheus on port 8080; pod monitor for blue/green
(nothing)	path plan	12 generated-file entries (`Deliverable`), including `apps/notes/namespace.yaml` and `estate/edge/public-frankfurt/notes-public.yaml`

Table 5: Declarations of `minimal`, Followed Through the Resolution

Four values depend on other run documents: the image needs the images lock, eligible machines need the node contract, the proxy needs the `edge Project`, and the collector address needs the observability `Project`.

Example 2 shows the authored `Process` in YAML, and Example 3 shows the `Process` the resolution assembled from it, in XMI.

```
- name: notes-api
  image: notes-api
  placement: { memory: 256Mi, cpu: 50m }
  startupBudget: 20s
  cutover: continuous
```

Example 2: The Authored `Process notes-api`, in YAML (Condensed)

```
<processes name="notes-api" deadline="60s" switchover="blue-green"
    image="ghcr.io/jorisjonkers-dev/notes/notes-api@sha256:9c1e5a4b..."
    uid="1000" gid="1000" memory="256Mi" cpu="50m">
  <startup period="5s" timeout="5s" failures="4"> ... </startup>
  <placement> <eligibleNodes>enschede-t1000-1</eligibleNodes> ... </placement>
</processes>
```

Example 3: The Resolved `Process notes-api`, in XMI (Abridged)

The path plan assigns the `Canary` resource, used by the Flagger rollout controller to manage blue/green replacement, to `apps/notes/notes/canary.yaml`. Application files follow `apps/<project>/<application>/`; here both names are `notes`. Section 4.1 explains the complete-model comparison.

3.4 The Derivation Rules

Table 6 lists the rules by target element. Each implements a derivation from the specification.

Target value	Rule	Derivation
Project		
reconcile unit and order	`unit`, `afterOf`	Create the `Project`'s unit. Its dependency list tells Flux to wait for provider projects' units and, where needed, the secrets-provisioning unit.
Application		
Secret Store	`synced`, `storeEndpoint`	The Secret Store (Vault) address used when a secret, backup or migration needs it.
migration	`migrationOf`, `datastoreOf`	Combine the locked runner image, schema version from the migration proof, and database address from dependencies; derive its credential and permitted connections.
release gate	`gate`, `switchesBlueGreen`	Register each blue/green `Process` with readiness as a gate member. Use the largest member deadline and platform analysis settings; enable error-rate and latency checks for members exporting telemetry.
Process		
image, user, group	`locked`	The lock entry for the image name.
switchover	`switchoverOf`	Use stop-start when interruption is permitted, rolling for `Application`s listed as platform delivery machinery, and blue/green otherwise.
deadline	`deadlineOf`	For a preparation job (lifecycle `prepare`), round the startup budget up to whole seconds without multiplying it. For other lifecycles multiply by three and round down; without a budget, use 600s.
probes	`Probe::cadenced`	Declared target with the period, timeout and failures from the `Platform Intent`. The startup probe polls the liveness target every 5s until the budget runs out.
eligible machines	`eligible`	Keep nodes meeting the declared placement requirements and CPU/memory requests, including sidecars. Continuous replacement requires capacity for both old and new copies.
environment	`environmentOf`, `entryFor`	Cluster-specific entries replace base entries of the same name. Resolve placeholders, add runtime-profile variables, and sort by variable name.
secrets	`Grant::resolvedFor`	A key-value `Grant` retains its path and keys and names the Kubernetes Secret to synchronise. A transit or database `Grant` records the Vault paths its policy permits.
volumes and backups	`backupFor`, `backupEgress`	For volumes requiring backups, select the platform schedule, retention and engine-specific image. Derive a separate service account, claim, database connection, credential and off-cluster destinations.
configuration files	`Asset::resolvedFor`	The file text, named after the `Process`, the file and a digest prefix, so a changed file gets a new name.
dependency edges	`edgeOf`	Select a `Process` providing the named `Application` and port; if none exists, try the platform's external providers. Record its address and permitted network peer.
Exposure		
entry point	`tierFor`	Select the first platform entry point whose allowed audiences include the `Exposure`'s declared audience.
route precedence	`rankOf`, `precedenceOf`	Exact routes come before prefix routes, and longer prefixes before shorter ones.
middleware	`middlewareOf`	Authentication for authenticated routes, security headers, and redirect if declared.
Network policy, per Process		

Target value	Rule	Derivation
ingress	`ingressOf`	Admit route proxies, the metrics collector for a scraped `Process`, dependency consumers, its backup identity and database migrations. For the Reporter (receiver of rollout events), also admit Flux's notification controller.
egress	`egressOf` and resolved connections	Add cluster DNS and, for secret holders, the Secret Store. Dependency edges supply provider peers; API rules supply server ranges. Migration and backup plans carry their own peers and off-cluster ranges.
Deployment		
provenance	`provenance`	Record each input's digest, then hash that record to identify the complete input set.
revision	end of deployment	Hash each `Application`'s complete resolved configuration in the specification's JSON representation (Section 3.6).
generated files	`projectFiles`, `applicationFiles`, `edgeFiles`, `vaultFiles`	Path of every file, with elements it is generated from (Section 3.4.5).

Table 6: Derivation Rules, by Target Element

3.4.1 The Process Mapping

Example 4 shows how the `Process` mapping combines these rules. The image lock entry supplies the repository and digest; `deadlineOf` supplies the deadline; `eligible` filters the nodes. Optional fields are assigned only where they apply. For example, a one-off job has no switchover: assigning the enumeration's default value would incorrectly record a blue/green deployment strategy for that job.

```
mapping PI::Process::process(in app : EffectiveApplication, in prj : EffectiveProject)
                : ResolvedProcess {
        init { var own : LockedImage := locked(self.image); }
        name := self.name;
        image := own.repository + '@' + own.digest;
        if self.lifecycle = Lifecycle::application
                then switchover := switchoverOf(app, self) endif;
        deadline := inSeconds(deadlineOf(self));
        secrets := self.secrets->collect(g | g.resolvedFor(self));
        placement := object ResolvedPlacement {
                eligibleNodes := contract.nodes->select(n | eligible(self, n))->collect(n
    | n.name);
        };
        dependencies := self.dependsOn->collect(e | edgeOf(e));
        environment := environmentOf(self, prj)->sortedBy(e | e.name);
}
```

Example 4: The Process Mapping, Abridged

3.4.2 The Edge Rule

A dependency edge names an `Application` and a `Surface` (named port). For example, an edge to `platform-postgres` names its `postgres` port. The query `edgeOf` finds the first `Process` in the cluster belonging to that `Application` that provides that `Surface`. The resolved edge records the `Process` address and the network peer the consumer may reach. If no `Process` provides the `Surface`, the `Platform Intent` external provider list is tried; this is how `auth` reaches a mail server outside the cluster. If neither has it, the run aborts.

During migration to this deployment system, the `Platform Intent`'s `HandoverLedger` records where applications still served by the previous system run. Their resolved addresses and peers use the recorded namespace and workload name until they move.

3.4.3 Environment Variables and Placeholders

A `Process`'s environment files consist of a base file and, optionally, one overlay per cluster. The resolution selects the overlay named by the `Platform Intent`'s cluster, then replaces base entries with overlay entries of the same name. Other base entries remain.

An entry may contain a placeholder, such as `DB_HOST=${dependency:platform-postgres.host}`. The query `Placeholder::entryFor` handles four kinds. A secret placeholder becomes a reference to a key of a secret the `Process` holds, so the value never appears in a file. A dependency placeholder takes the host or port of the `Process` dependency edge from `edgeOf`. An exposure placeholder takes the URL, host or scheme of an `Exposure` anywhere in the cluster. An identity placeholder takes the `Process` name, namespace or Vault role (the binding that grants its service account a secret-access policy). Text after a placeholder is kept, as in `AUTH_LOGIN_URL=${exposure:auth.public#url}/login`.

The runtime profile then adds the platform-defined variables for the selected language runtime, such as `PORT`. Declaring one of these names in an environment file aborts the run, because it would conflict with the platform-defined value.

3.4.4 Network Peers

The ingress policy of a `Process` combines peers from six sources (Table 6). Most require looking across `Project`s: consumers with a dependency edge to the `Process`, migrations that access its database, proxies and the metrics collector named by the `Platform Intent`. Each source yields tuples of rule, namespace, `Process` and port. Distinct `PolicyPeer` objects would not compare equal merely because their fields match. The union therefore uses tuples and `asOrderedSet()` to remove duplicates before creating `PolicyPeer` objects. The rule name remains part of each tuple, so two rules allowing the same connection retain both rule identifiers.

3.4.5 The Path Plan

The path plan represents each generated file as an instance of a `Deliverable` subclass, recording its path and the model elements used to generate it. Each `Project` gets a namespace file, index and default-deny network policy. If an `Application` synchronises secrets, a `VaultConnection` file tells the secret operator which Vault address to read.

Each `Application` gets a directory with one file per resource kind: a workload file, a service account, RBAC, volume claims, a configuration map carrying `ReleaseGate` inputs as JSON, backups, a migration, a monitor, synced secrets, and a network-policy file with one policy per `Process`. Blue/green `Process`es add a `Canary` file, others a `Service` file; multi-replica `Process`es add a disruption budget. Empty files are not planned.

Each index lists the files Flux should apply from its directory. Network policies are included in the generated output for review, but excluded from these indexes while the specification's policy stage is render-only. This allows their proposed connections to be inspected before enforcement. Each `Exposure` gets a route file under its entry point (Traefik listener). Each service account holding a secret gets a Vault policy and role.

Paths can therefore be inspected and compared before rendering; the transformation has no separate path-collision abort. Templates receive these paths rather than choosing them.

3.5 The Trace and End Sections

Gate members, routes, metrics scrapes and per-`Process` files refer to a `ResolvedProcess`. They find it by calling `p.resolveone(ResolvedProcess)` on the source `Process`, which the `Application` mapping has already transformed (Section 3.2).

The `end` sections set values whose prerequisites are created by the mapping body. The `Revision` must wait until the `Application` is complete; the migration job tag then takes a prefix of that digest so both identify the same release. Similarly, the render hash is computed after all input digests have been recorded, and `ReconcileUnit`

dependencies are assigned after their local reference objects exist. Each output contains its own unit and name-only `ReconcileUnit` objects for prerequisite units in other projects; `after` points to those local objects, not into another output model.

3.6 The Black-Box Library

A digest is a SHA-256 hash over canonical text. OCL lacks a byte-hashing operation, so the Java class `Hashing` provides three contextual operations. `digestOf` walks any model element reflectively, writes canonical JSON and hashes it for input provenance. `textDigestOf` hashes a file's text, for the content-derived names of configuration files. `rawDigestOf` hashes a string as-is.

The `Revision` uses `rawDigestOf`. Generated files carry this digest, so it must match the oracle exactly. The specification hashes the `Application` configuration in canonical JSON (RFC 8785). That specified JSON has a different structure from the Ecore target model. The OCL query `elementJson` therefore constructs the required JSON from the source and target models, sorting keys, omitting absent values and using the specified representation of each value. The black box only hashes this text; the transformation determines which values it contains.

Calls explicitly cast model elements to `EObject` before invoking the library's contextual operations. The standalone runner and Eclipse plug-in both register the `Hashing` library; the QVTo modules access it by import.

3.7 The Final Pass

One job installs the Vault policies and roles from every `Project` into the Secret Store. Its name contains a digest of all these documents. Changing a document therefore gives the next deployment a new job name, causing the updated configuration to be installed.

The final pass, `estate(inout rendered : RD)`, runs once over all `Resolved Deployment`s produced together. It collects every `VaultPolicyFile` and `VaultRoleFile`, writes each as canonical JSON in OCL, sorts them by file name, hashes the whole with `rawDigestOf`, and plans the job, configuration map, service account and network policy. These files and their index are stored in the first model in the caller's project list, so the job is generated once for the combined output. The pass also plans one cluster-wide route forwarding Flux events to the Reporter. All resolutions normally share one `Platform Intent`. A defensive assertion rejects different job namespaces or images if the pass is called directly with models from different runs; it does not compare every job setting.

3.8 Aborts and Running the Transformations

Fifteen fatal assertions in the resolution abort the run when a value cannot be derived. Thirteen are in abort queries that a rule calls instead of returning an invalid value. For example, `locked` returns the lock entry for an image or calls `unlocked`, whose assert fatal ends the run with `E_UNLOCKED_IMAGE`. Two check `ReleaseGate` preconditions at the start of the gate mapping. The final pass has one more, for the inconsistent job namespaces or images described above.

The checks reject missing inputs and providers, unsupported quantities, unresolved placeholders, invalid secret references and author-defined runtime-profile variables. They also reject backups without schedules or engines, migrations without a database, blue/green deployments without readiness or delivery services, and `Grant`s without a Secret Store.

Where the specification defines an error code, the message contains it. An audience unsupported by any entry point needs no abort here: the `E_NO_TIER_FOR_AUDIENCE` constraint already rejects it before the transformation runs.

Three Java classes (`Lowering`, `Resolution` and `Estate`) run the transformations through QVTo's own `TransformationExecutor`. They register the black box, set configuration properties, pass the extents, and return the result or fail with the executor's diagnostic. `Lowering` is the implementation name of the flattening runner; this pass does not change the abstraction level.

The build executes the transformations with QVTo's standalone executor on every code change. The archive's `README.md` gives terminal and Eclipse instructions for all three passes. It also maps each example to its output project and explains the generated model extents.

4 Tests and Evaluation

The transformations are tested by running them on eight examples and comparing their output with oracle files: the expected outputs committed with the specification's worked examples. Agreement supports conformance to those expected values; the strength of that evidence depends on how each expected result was obtained.

The build runs six test types on every change.

- **Flattened models.** `auth`, `data`, `knowledge-platform` and `minimal` are compared byte-for-byte with `expected/effective.json` beside each specification example. These committed fixtures are not regenerated during the tests.
- **Dependency edges.** For each example, the resolved edges are exported as JSON and compared byte-for-byte with the expected file. These tests read only files the pipeline wrote, so they check the pipeline as a user would run it.
- **Revisions.** Each expected resolved file records its `Revision`, a digest of the `Application` element in the specification's canonical JSON form. A matching `Revision` checks the represented decisions, including each `Process`'s ingress and egress peers (Section 4.1.1).
- **Complete expected models.** The adapted Task 1 model of `minimal` and a reviewed regression snapshot of `secrets` are compared using EMF's structural equality (`EcoreUtil.equals`). No attributes or references are excluded. Both expected models record the transformation's provenance digests; matching those fields checks stability, not agreement with the specification's differently represented inputs.
- **Generated files.** Tests run the Task 3 templates on the transformation output and compare every generated file with its committed counterpart. This checks the resolved values used by the templates as well as their serialization.
- **Interface tests.** JUnit tests check that the resolution accepts a minimal `Project`, rejects a `Project` absent from the input, and reports the file name when a configuration file is missing. Final-pass tests check that one Vault policy job covers two `Projects`, its name changes when a document changes, no job is generated without documents, and conflicting job definitions abort the run. Eight flattening cases check the merging of shared declarations.

Coverage and mutation tools require full line and branch coverage and a 100% mutation score for the Java integration code. They do not measure QVTo, so the comparisons establish no QVTo branch-coverage result.

4.1 Results

Table 7 lists the eight examples. All resolve successfully and every comparison passes, including the complete-model comparison of `minimal` and `secrets`. The full build passes 323 tests. Its oracle suite contains 81: 10 parsed-intent comparisons, four flattened models, 47 input refusals, eight composition refusals, eight dependency lists, one rendered tree, one metamodel descriptor and two checks on the test setup. This total includes Task 1 parsing and constraint tests.

	Counted in the resolved model							Equal to the expected output			
Example	**Apps**	**Proc.**	**Edges**	**Secr.**	**Vol.**	**Env.**	**Files**	**Edges**	**Revisions**	**Files**	**Model**
`minimal`	1	1	0	0	0	6	12	empty	1/1	yes	yes
`auth`	1	2	4	3	0	21	19	yes	1/1	yes	–
`data`	3	3	0	1	3	3	37	empty	3/3	yes	–
`delivery`	4	4	1	0	0	0	27	yes	4/4	yes	–
`edge`	2	2	0	0	0	0	13	empty	–	–	–
`knowledge-platform`	4	7	5	4	6	84	49	yes	1/4	yes	–
`observability`	2	2	0	0	0	0	15	empty	–	–	–
`secrets`	1	1	0	0	0	0	8	empty	–	–	yes

Table 7: The Resolved Examples and Their Comparisons (–: no committed oracle of that kind)

"Empty" means the expected dependency lists contain no edges; only `auth`, `delivery` and `knowledge-platform` compare non-empty edge sets. Revision fractions give matching digests over the total number of `Applications`, so three of `knowledge-platform`'s four have no oracle digest comparison.

Between them, the examples exercise the derivations in Table 6. `minimal` exercises the common `Process` and `Exposure` rules (Table 5). The migration, placeholder and external-provider rules run in `auth`, which holds

three secrets and reaches a mail server outside the cluster. The volume, backup and configuration-file rules run in `data`, which holds one volume of each durability class (reconstructible, recoverable, irreplaceable), a `Sidecar` and a configuration file. The rolling and stop-start switchover rules run in `delivery`, which also has API access. `secrets` models the Secret Store, `knowledge-platform` is the largest example, and `edge` and `observability` supply the proxies and collectors that other examples look up.

4.1.1 Revision Evidence

An `Application`'s canonical JSON includes its `Process`es with images, identities, probes, placement, secrets, volumes, backups, environment, dependencies and network peers; its `Exposure`s and routes; its `ReleaseGate`; and its migration. Provenance, file paths, typed resource objects and the estate-wide policy job fall outside it.

The transformation builds that text from its own `Resolved Deployment` (Section 3.6), so one different value, one extra environment variable or one different order changes the digest. Ten `Application`s across five examples have a `Revision` in an expected file, and all ten match.

The comparison checks this representation as a whole, without locating a mismatch or checking omitted values. A further test checks sensitivity: increasing memory in `minimal` from 256Mi to 512Mi changes `Revision`, whereas changing only the capture time of the `ClusterState` snapshot leaves it unchanged.

4.1.2 Generated-File Evidence

Generated-file comparisons additionally check typed resource serialization, Vault documents and destination paths. Although `Revision` covers a `Process`'s network peers, it does not check their serialization as Kubernetes policies. Across five examples and the cluster-wide output, all 167 committed files are reproduced byte-for-byte, including the final-pass job files.

4.1.3 Gaps in the Evidence

Two examples, `edge` and `observability`, have no `Revision` oracle, no committed generated files and no expected model. Their `dependencies.json` files list no edges. Evidence for them is therefore limited to successful execution and comparisons for examples that use their proxies and collectors. Those check the values consumed by other `Project`s, leaving the remaining configuration unchecked.

The `secrets` snapshot has no independent JSON counterpart or committed generated files. Reviewing the transformation's output to obtain that snapshot found a specification gap: the Secret Store network policy allows no inbound traffic, although holders of secrets are allowed to send traffic to it. If enforced, the receiving policy would still block those connections; permission on the sending side is insufficient.

The Task 1 model of `minimal` was adapted to the metamodel changes in Appendix A.2. Its `Revision` was copied from the oracle, so that field and its recorded provenance provide regression checks rather than independent derivations.

4.1.4 Defects Found

One directory-layout defect required a correction: cluster-wide route files were planned under `apps/edge/`, inside the `edge Project`'s directory. This mixed ownership of a directory without requiring two files to claim an identical path, so duplicate-path checks could not detect it. Cluster-wide files moved to `estate/`; the specification's path rules, path plan and expected files changed together. Agreement after that correction guards against regression. It is not an independent test of the revised path rules, which required reviewing who consumes each directory.

4.2 Limitations

Agreement with the oracles supports the transformation's conformance to their expected outputs. It does not establish that the oracles or the specification are correct. The hand-written Task 1 model of `minimal` supplies a partly independent check; its adapted fields have the qualifications above.

The examples exercise the derivation families, but a matching result for one input does not prove that every branch works. The `knowledge` example was excluded because that `Project` is being retired and its successor

knowledge-platform was selected instead. The suite does not attempt its resolution, so exclusion supplies no evidence that it succeeds or that retirement is unsupported.

During Task 2, checks for secret references, placeholders and platform requirements were added to project-intent.ocl, before resolution; these errors had previously reached QVTo aborts. Remaining aborts stop at the first problem without a source location. Cross-document constraints still use allInstances(); navigation from a single root object remains planned.

Placement produces eligible nodes for each Process; Kubernetes chooses the actual node. Each eligibility check considers that Process in isolation, so several may be eligible for a node even though their combined requests exceed its capacity.

Ordering between ReconcileUnits is defined at Project granularity because Flux orders these groups of files as whole units; individual Applications within a unit cannot be ordered separately.

Provenance uses the EMF representation of inputs, so its digests differ from the specification's. Whole-model comparisons pass because the expected EMF models record those computed values, not because provenance is omitted. Digests used in generated files do match: Revision, configuration-file names and the Vault policy job name.

Automated tests use the standalone executor. The Eclipse launch configuration and plugin.xml registration have only been checked manually.

4.3 Reflection

Flattening made resolution rules simpler: each reads one selected value from a Process, without repeating the precedence rules for shared declarations. It also introduced an intermediate model and copying rules that need their own tests. A failed merge comparison can be investigated without involving image or provider resolution. This separation makes the extra model worthwhile.

Resolving one Project at a time kept its result inspectable, but did not remove the need to reason about the estate. Cross-project references still require the full input extent, and a digest over all Vault documents requires the final pass. The split therefore helped local comparison at the cost of another transformation and a rule for where shared output is stored.

QVTo's explicit sequencing suited creation of Processes before their routes and computation of Revision last. Its trace avoided a manual source-to-target table. ATL also offers traces and explicitly invoked rules, so these results support the chosen organisation without establishing that ATL could not express it. No equivalent ATL implementation was built.

The greatest effort went into reproducing Revision. The target model has a different structure from the specification's JSON, so the transformation had to reconstruct the specified field names, omissions and ordering before hashing. That reconstruction occupies about one sixth of resolution.qvto. A hash mismatch could mean either a wrong deployment decision or a wrong JSON representation, with no indication of where they differed. A better test design would compare that JSON directly before checking its digest, separating derivation errors from serialization errors. Using a Java hash function saved little of this work: most of the code was needed to construct the JSON in OCL.

Evidence should have been planned alongside the examples. secrets and knowledge-platform ran before their output comparisons were added; those runs established execution, not correctness of the derived values. Constructing expected results before running the transformation, and retaining the original Task 1 model beside its adapted version, would make independent checks easier to distinguish from regression snapshots.

References

[1] P. Merle and F. Petrillo. Visualizing Cloud-native Applications with KubeDiagrams. In *Proc. VISSOFT 2025*, pp. 106–116, 2025. Tool: https://github.com/philippemerle/KubeDiagrams.

[2] KubeOrch, visual Kubernetes workflow orchestrator. https://github.com/KubeOrch (repositories core and ui), visited 15 October 2026.

[3] kubert, a kubectx and kubens alternative. https://github.com/idebeijer/kubert, visited 15 October 2026.

[4] Object Management Group. *Meta Object Facility Query/View/Transformation Specification*, version 1.3, 2016.

A Changes to the Task 1 Metamodels

This appendix lists the changes between the Task 1 metamodels and those the Task 2 transformations read and write. The comparison checked each class and feature in the `.ecore` files. The diagrams are grouped at the end of this appendix.

A third metamodel for the `Pinned input set`, the read-only documents no `Project` author writes, was added (Appendix A.1). The target metamodel gained derived values and twelve kinds of generated file (Appendix A.2), and the source metamodel gained declarations the other examples need (Appendix A.3). Four grammars were added (Appendix A.4).

No class was removed, but some features were removed or replaced, including document headers on `EffectiveProject` and single-`Process` references on generated-file classes. The Task 1 hand-in showed OCL helper definitions such as `levelsAbove` and `derivesDatabase` as derived attributes in the diagram. They remain OCL definitions in `project-intent.ocl` and were never Ecore features.

A.1 The Pinned Inputs Metamodel

The `pinnedinputs` package models the additional documents read by the resolution (Section 1.3).

The node contract (the list of machines and their capacity) is generated from the platform team's machine descriptions. Other programs publish the images lock (each image pinned to a digest) and the cluster-state snapshot. A `Project` build writes its migration proof (the schema version its migration was tested against).

A stored JSON description of the source metamodel's classes and features is compared with the specification's authored-document schemas. Pinned inputs are kept in a separate metamodel because they are not part of those schemas. The target metamodel has no references to pinned-input classes: the `Resolved Deployment` records the values derived from these inputs and their digests instead.

Figure 3 shows the metamodel. Table 8 shows which rules read each class.

Class	Read by
`NodeContract`, `Node`, `Allocatable`, `Gpu`, `Disk`	`eligible` compares resource requests with available capacity and checks hardware requirements. CPU quantities such as 53500m denote 53.5 cores.
`ImagesLock`, `LockedImage`	`locked`: every image of a `Process`, `Sidecar`, backup method, migration runner and Vault policy job.
`ClusterState`, `Binding`	`boundOf`: the machine that already holds a `Volume`.
`CurrentPlacement`, `RollbackBackup`	Retained as snapshot data for delivery operations; these classes do not affect the resolution's output.
`MigrationProof`, `ProvenApplication`	`migrationOf` records the tested schema version; `gate` also reads whether the release contains non-transactional changes.
`AssetFile`	`Asset::resolvedFor` reads the configuration-file text. Each file is wrapped in an `AssetFile`, rather than parsed with an Xtext grammar.

Table 8: The Classes of the Pinned Inputs Metamodel and the Rules That Read Them

Disk medium (field `media`) and architecture are strings in the node contract, although the source metamodel restricts `Process` requests to enumerations. The contract can describe hardware, such as an sdcard, that no `Process` may request. Reusing the source enumeration would prevent such nodes from being represented. Eligibility therefore compares each requested enumeration's spelling with the corresponding contract string.

A.2 Changes to the Target Metamodel

Both halves of the target metamodel grew: the values the resolution derives for each `Application`, and the generated-file entries of the path plan. Derivation rules read policies from the `Platform Intent`; the target metamodel records the resulting values.

The root class is unchanged, but there is now one instance per `Project` instead of one for the whole cluster (Section 1.3).

Derived Values. Figures 4 and 5 show the new or changed derived-value classes at application and process level. Table 9 lists their feature changes and reasons.

Task 1 described many values in prose but modelled only those needed by its worked example. The other examples also need migrations, backups with database connections, secrets synchronised into the cluster, API access, `Sidecar`s, configuration files, and two cluster-wide values: the Vault policy job, which writes every Vault policy, and the route of Flux's events.

Class	Change	Reason
`ResolvedDeployment`	added `policyJob`, `events`	The Vault policy job and the route for Flux's events belong to the whole cluster; every `Project` model carries the same job; the final pass plans their files.
`ResolvedPolicyJob`, `ResolvedEvents`	new	The policy job records its identity, image, role, resources and egress. The events route records the Reporter's namespace and address.
`ResolvedApplication`	added `secretStore`, `scrape`	Record the Vault address for secret synchronisation and the metrics scrape settings. Both contribute to the `Revision`.
`ResolvedScrape`	new	The scraped `Process`, `Surface` (named port), path, interval and timeout.
`ReleaseGate`	added `endpoint`, `migration`	An address that every `Canary` (Flagger) webhook calls; the migration the gate starts and may undo.
`GateAnalysis`	added `errorRate`, `latency`	The thresholds of the two analysis checks, from the `Platform Intent`.
`GateMigration`	new	The migration's service account and what its proof records.
`ResolvedMigration`	added `uid`, `gid`, `identity`, `deadline`, `memory`, `cpu`, `scratch`, `database`, `credential`, `egress`	Task 1 named the service account, owner role and deadline in prose. A migration job also needs the database address, the owner credential that only it holds, and a network policy.
`MigratedDatabase`	new	The database host, port and name.
`ResolvedProcess`	added `lifecycle`, `api`, `assets`, `sidecars`	Record whether the `Process` serves continuously, runs once or prepares a release, plus its API access, configuration files and `Sidecar`s. These values contribute to the `Revision`.
`ResolvedApiAccess`, `ResolvedApiRule`, `AdmittedRange`	new	The API rules a `Process` holds; the address range a policy admits: the API server for API access, the backup off-cluster destination.
`ResolvedAsset`, `ResolvedSidecar`	new	A configuration file with a content-derived name; a `Sidecar` with a locked image and resources.
`ResolvedGrant`	`path` and `access` optional; added `engine`, `paths`, `destination`, `mountAt`, `fileMode`	A transit or database `Grant` covers several Vault paths. A synchronised key-value `Grant` also records the destination Kubernetes Secret and its mount settings.
`PolicyPath`	new	One path of a Vault policy and the operations allowed on it.
`ResolvedVolume`	added `mountAt`	The workload template needs the mount path.
`BackupPlan`	added `uid`, `gid`, `identity`, `claim`, `egress`, `peer`, `credential`, `destinations`	A backup runs as its own service account with its own claim and network policy; it connects to the `Process` it dumps and copies off-cluster with a credential.

Class	Change	Reason
`BackupPeer`	new	Where a backup connects, and the credential it logs in with.
`EnvEntry`	`value` optional; added `secret`	A secret placeholder becomes a reference, so the value never appears in a file.
`SecretReference`	new	The secret path and key a variable reads.
`StartupProbe`	added `timeout`	The specification gives the startup probe the `Platform Intent` probe timeout.
`ResolvedExposure`	added `listener`, `certificates`, `proxyApplication`, `proxyNamespace`	The entry point's protocol and certificate source; the proxy whose middleware the routes name.
`PolicyPeer`	added `rule`; `process` optional	A peer identifies a permitted network connection. Recording its rule preserves distinct reasons for allowing the same connection. Cluster DNS needs a namespace but no `Process`.

Table 9: Changes to the Derived-Value Classes of the Target Metamodel

Generated Files. Figures 6 and 7 show every subclass of `Deliverable`, the generated-file entry, up from Task 1's nine to 21.

Workload, Canary and service account files, which Task 1 tied to one `Process`, now list all of an `Application`'s `Processes`, because the specification puts one file per `Application` holding all its `Process` resources. For the same reason, the network policy file holds one `ProcessPolicy` per `Process`, and the Secret file refers to its `Application` and holds one `SecretHolder` per service account, since a backup's service account holds Secrets too. Task 1's worked example had one `Process`, so this difference did not show.

`MonitorFile` gained timeout and kind: a stop-start `Process` is scraped through its `Service`; other strategies use a Pod monitor.

The twelve new file kinds are Services, volume claims, configuration maps, backups, disruption budgets, migrations, RBAC, the `VaultConnection` used by the secret operator, Vault policies, Vault roles, the Vault policy job and the events route.

The hand-written model of `minimal` from Task 1 was updated with these changes. Its files refer to the `Process` through `processes`, it gained the configuration map that carries the `ReleaseGate` inputs, and its expected values were adapted as discussed in Section 4.1.3.

A.3 Changes to the Source Metamodel

The source metamodel kept every class. Table 10 lists what was added or changed.

Most additions make platform policies explicit that Task 1 left implicit. Declaring these values in the `Platform Intent` keeps them out of the transformation rules, where they would otherwise be hard-coded.

Class	Change	Reason
`Process`	added `api` (`ApiAccess`, `ApiRule`, `ApiVerb`)	The `Process` declares the Kubernetes API access it needs, with a reason; the platform's own delivery `Applications` need it.
`Platform`	added `apiAccess` (`ApiAccessPolicy`)	Which `Applications` may hold API access, and the API server address range.
`Platform`	added `secretStore`, `telemetry` (`TelemetryPolicy`), `policyJob` (`VaultPolicyJob`)	References identify Vault and the telemetry services in other `Projects`. The policy job declares its image, role and resource requests.
`DeliveryPolicy`	added `gate`, `reporter`	References to the Release Gate service and Reporter `Applications`.

Class	Change	Reason
`AnalysisPolicy`	added `errorRate`, `latency`	The thresholds the Release Gate service checks.
`Substrate`	added `clusterDns`	The namespace every egress policy admits for DNS.
`EnginePolicy`	added `surface`	The port a database engine backup connects to.
`OffClusterCopy`	added `egress` (`DestinationRange`)	The address ranges a backup copy goes to.
`HandoverLedger`	added `serving` (`ServingLocation`)	Where a `Project` not yet moved still runs, so an edge to it resolves.
`RenderedArtifacts`	added `pullSecret`	Used by delivery, not by the resolution.
`Placeholder`	added `suffix`	Text after a placeholder, as in `${exposure:auth.public#url}/login`.
`Grant`	`rotation` required	Every `Grant` states how it tolerates rotation, which decides its restart targets.
`Volume`	`size` required	A claim cannot be generated without a size.
`EffectiveProject`	removed `apiVersion`, `kind`, `schemaVersion`	The flattened model is written by the flattening, not by an author, so it has no document header.

Table 10: Changes to the Source Metamodel

A.4 Grammars for the Pinned Inputs

The pinned inputs use grammars rather than a Java YAML reader or hand-written XMI, so the transformation can read the examples' YAML directly. One quoting adjustment was needed, as described below; no input value changed.

Four grammars were added beside the Project and Platform grammars: NodeContract, ImagesLock, ClusterState and MigrationProof. Each inherits the Project grammar, reusing its terminals, indentation tokens and scalar rule, and adds only the rules for its own document. As with the `Platform Intent`, the file name selects the grammar: `node-contract.yml`, `images.lock.yml`, `cluster-state.yml` and `migration-proof.yml`.

One input file changed: the Project grammar scalar terminal cannot contain a colon, so the snapshot's capture time is now quoted, as digests and URLs already were. The value is the same either way.

Example 5 shows the smallest grammar.

```
grammar dev.jorisjonkers.deploykit.emf.syntax.ClusterState
        with dev.jorisjonkers.deploykit.emf.syntax.ProjectIntent
import "https://jorisjonkers.dev/deploy-kit/pinned-inputs/1"

ClusterState returns ClusterState:
        (('apiVersion' ':' apiVersion=Text)
        & ('kind' ':' kind=Text)
        & ('schemaVersion' ':' schemaVersion=Text)
        & ('cluster' ':' cluster=Text)
        & ('capturedAt' ':' capturedAt=Text)
        & ('bindings' ':' BEGIN (DASH BEGIN bindings+=Binding END)* END)
        & ('placements' ':' BEGIN (DASH BEGIN placements+=CurrentPlacement END)* END)
        & ('backups' ':' BEGIN (DASH BEGIN backups+=RollbackBackup END)* END)?);

Binding returns Binding:
        (('claim' ':' claim=Text)
        & ('node' ':' node=Text));
```

Example 5: The Grammar of the Cluster-State Snapshot, Abridged

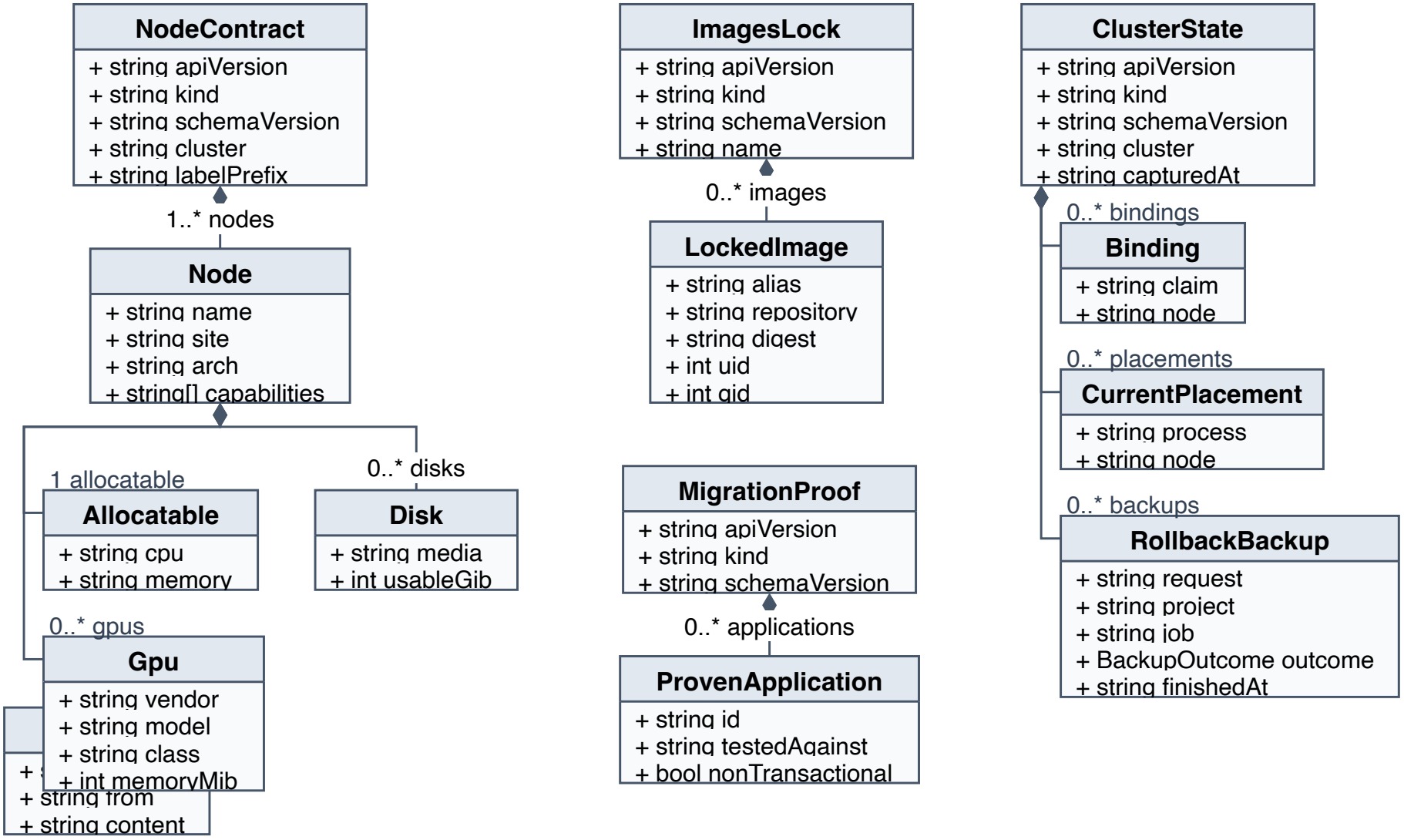


Figure 3: The Pinned Inputs Metamodel. All classes are new; diamonds mark containment

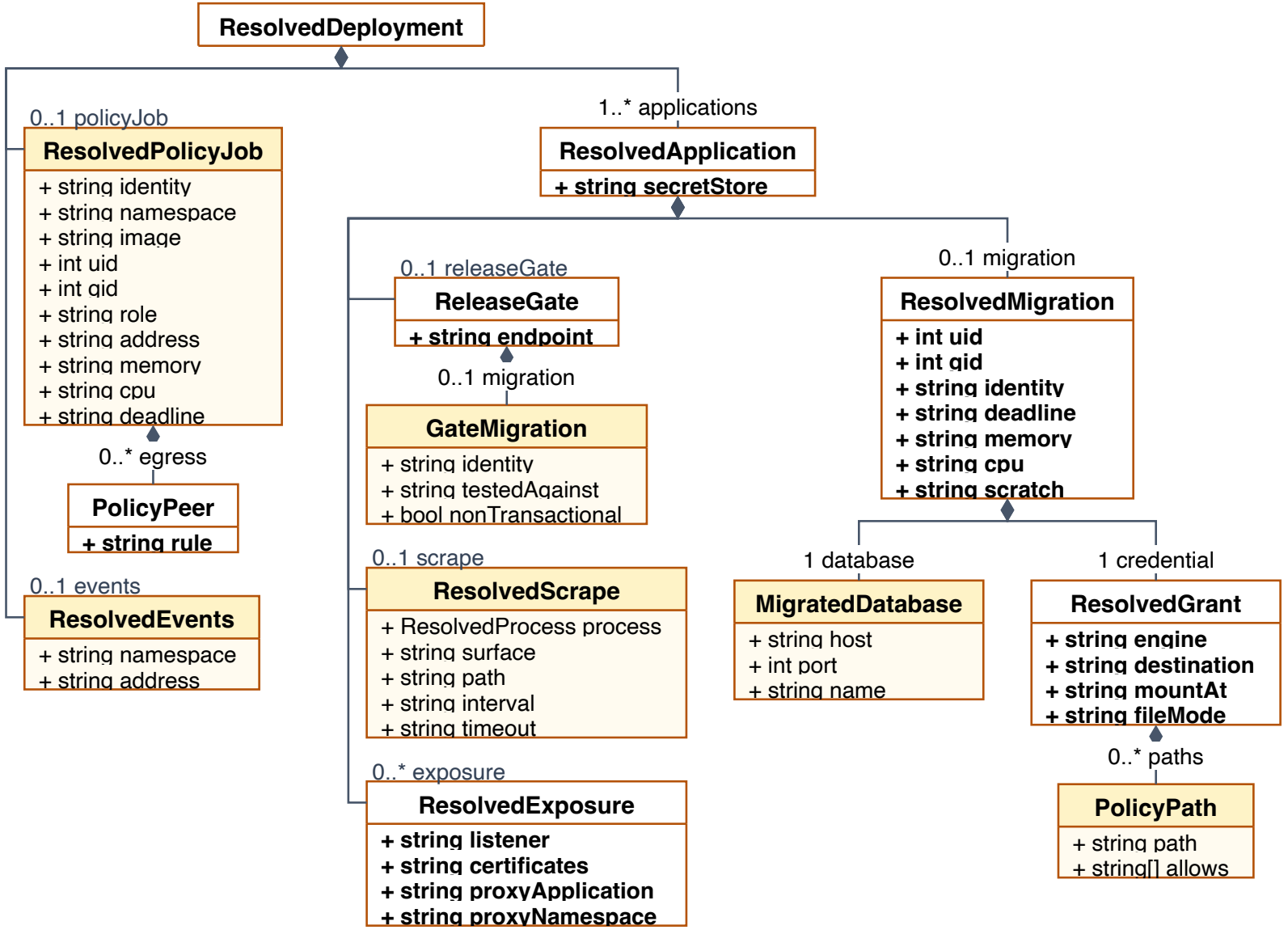


Figure 4: Changed Deployment and Application Classes. Filled classes are new; white Task 1 classes show added features in bold. Diamonds mark containment

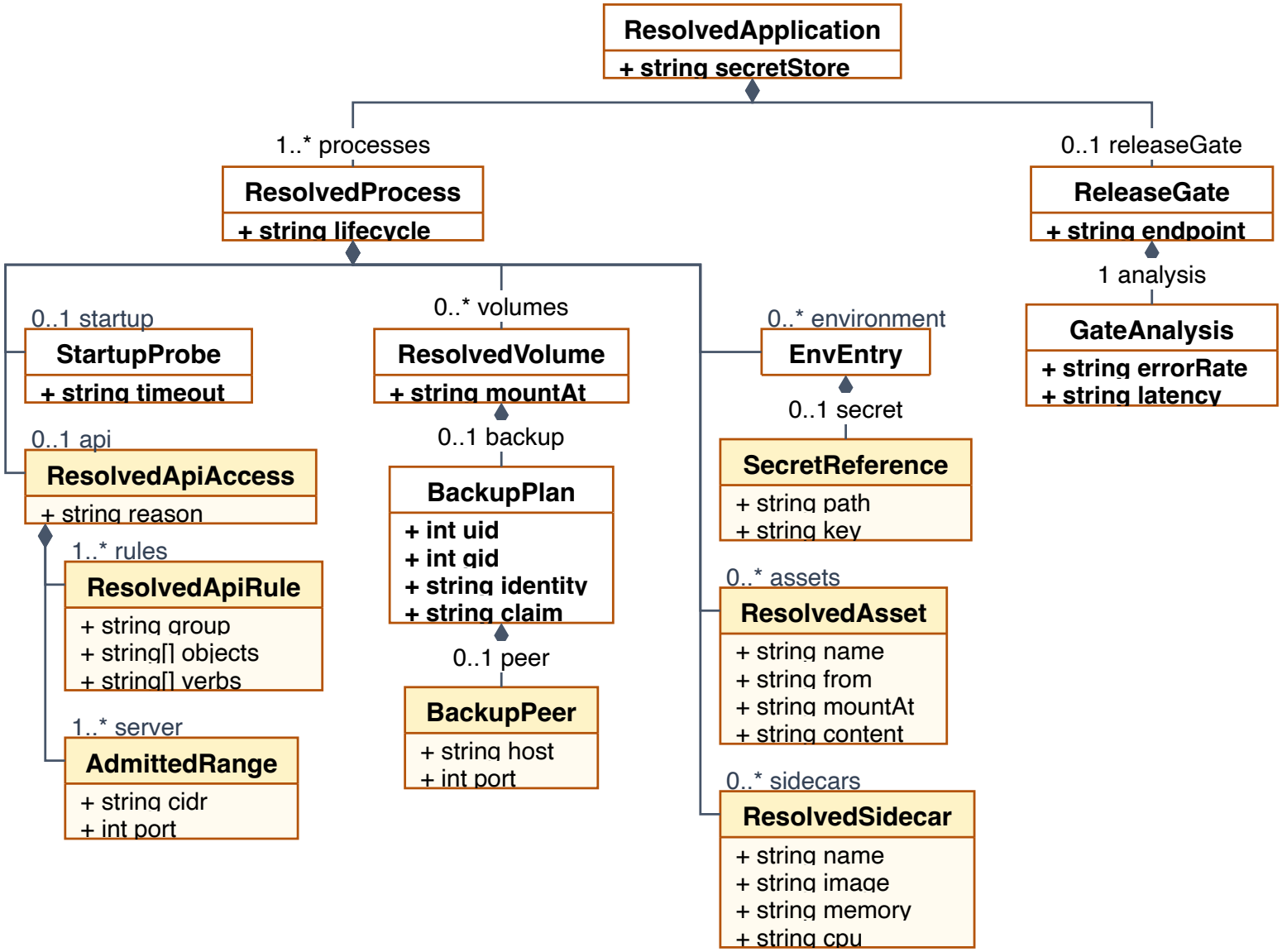


Figure 5: Changed Process and Release Analysis Classes. Parent classes are repeated for context; colours and feature markings follow Figure 4

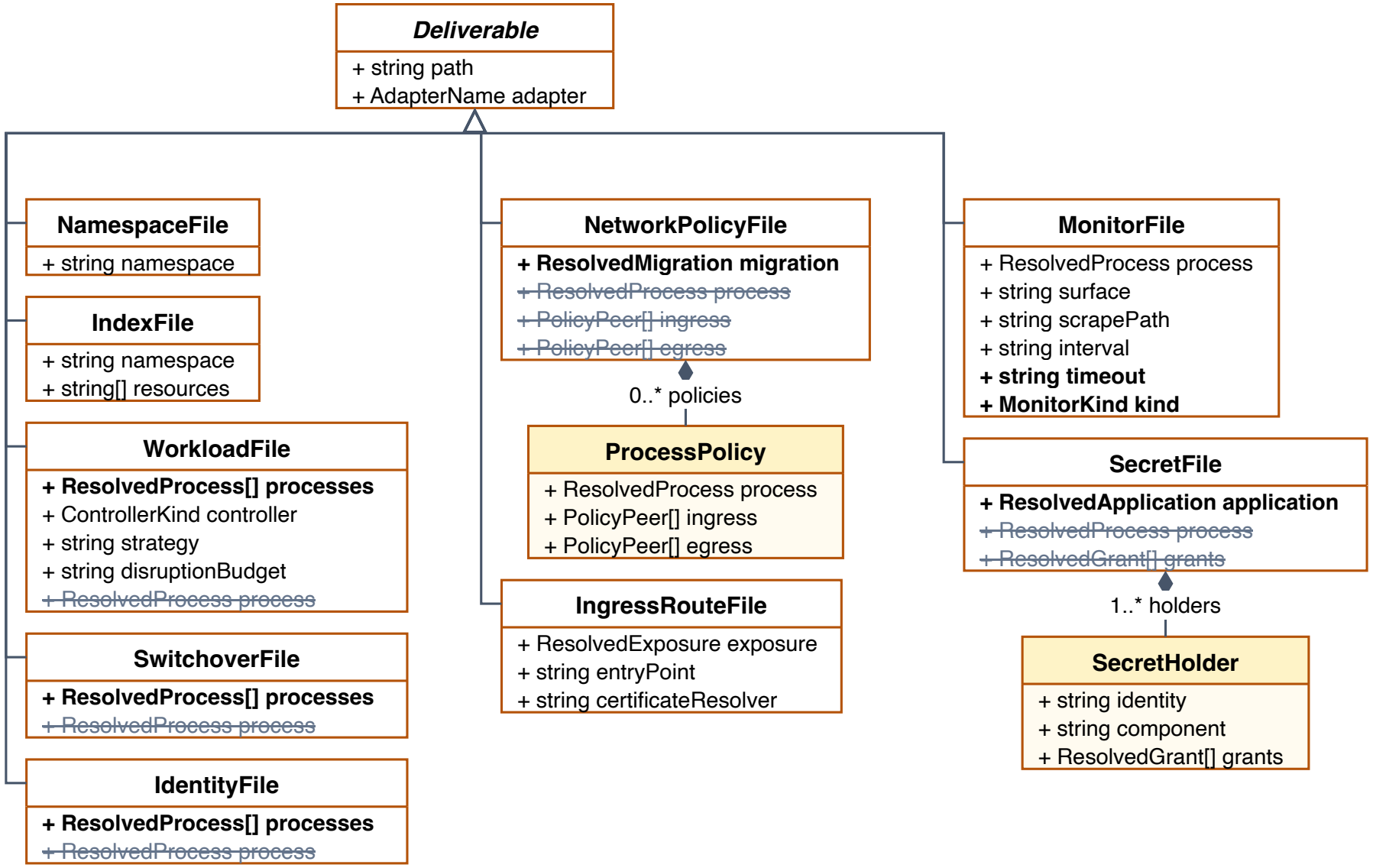


Figure 6: Task 1 Generated-File Classes and Their Changes. Bold marks added features; strikethrough marks removed features. Filled helper classes are new

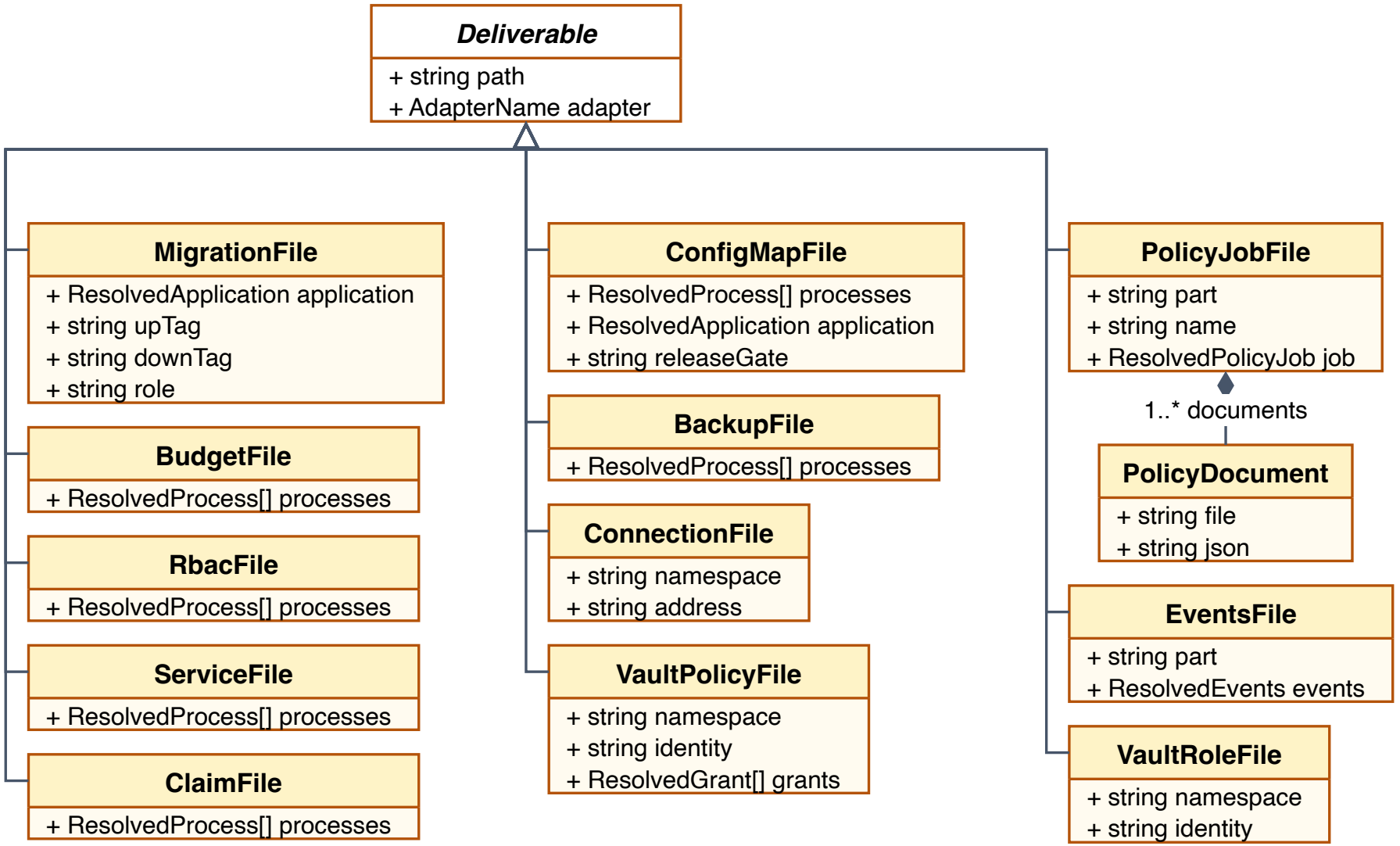


Figure 7: The Twelve Added Generated-File Classes. The Abstract Deliverable Parent is Repeated for Context; All Classes Belong to the Target Metamodel